

Esempi di SO

- Linux scheduling
- Windows scheduling

Scheduling Linux prima della versione 2.5

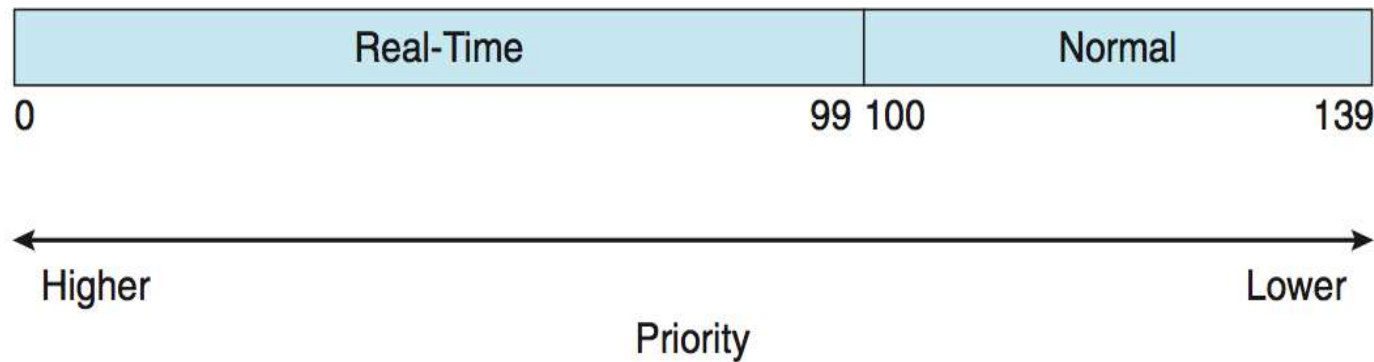
- Prima della versione kernel 2.5 variazione dell'algoritmo di scheduling standard di UNIX
- Linux v1.2
 - Unica lista contenente tutti i processi attivati sulla macchina.
 - ▶ Il primo elemento della lista è la task_struct di init
 - Ogni processo ha una priorità statica (nice value o nice)
 - La priorità varia da -20 (priorità più alta) a +19 (priorità più bassa).
 - Il valore di default è 10.
 - Schema classico delle priorità in UNIX.
 - Scheduling in modalità “**round robin pesato**”
 - ▶ Un processo in attesa da più tempo viene favorito rispetto ad un processo che attende da meno tempo. A parità di attesa, un processo a priorità più alta viene favorito rispetto ad un processo a priorità più bassa.
 - ▶ Ad ogni invocazione dello schedule, la lista è scandita ed i contatori aggiornati, si sceglie il processo per cui contatore è massimo

Scheduling Linux prima della versione 2.5

- Prima della versione kernel 2.5 variazione dell'algoritmo di scheduling standard di UNIX
- Linux v2.2
 - **Classi priorità** multipla
 - Priority-based e time-sharing
 - Prelazione con priorità
 - Funzione goodness() ritorna un peso (weight) nell'intervallo [-1000, 1000].
 - ▶ Dipende dalla priorità e dai tempi di esecuzione
 - Il peso è usato per scegliere il processo da schedulare.
 - ▶ Selezione basata su Priorità con RR (se stessa goodness)
- Si usano 140 livelli di priorità.
 - Livelli [0, 99]: usati da algoritmi di scheduling di tipo soft real time (SCHED_FIFO, SCHED_RR), che hanno sempre la precedenza sull'algoritmo standard.
 - Livelli [100, 139]: corrispondono all'intervallo [-20, 19] (SCHED_OTHER) e sono utilizzati dall'algoritmo di scheduling di default (round robin pesato).

Linux Real-Time Scheduling

- Real-time scheduling in POSIX.1b con SCHED_FIFO o SCHED_RR
 - Task real-time assegnati a priorità statiche tra 0 e 99
 - Real-time hanno valori più bassi di quelli dei task normali e sono mappati in uno schema di priorità globale



- Le priorità relative dei processi real-time sono assicurate
- Il kernel non fornisce garanzia sui tempi di attesa dei processi pronti
- Se interruzione per real-time arriva mentre il kernel serve una chiamata di sistema, il processo real-time deve attendere

Scheduling Linux prima della versione 2.5

- Prima della versione kernel 2.5 variazione dell'algoritmo di scheduling standard di UNIX
- **Classi priorità** gestite con RR pesato
 - Priority-based e time-sharing
 - Prelazione con priorità
 - Priorità assegnata sulla base di una funzione goodness() ritorna un peso (weight) nell'intervallo [-1000, 1000].
 - Il peso è usato per scegliere il processo da schedulare.
 - Uso di quanti di tempo variabili (time slice).
 - L'algoritmo di scheduling è $O(n)$ rispetto al numero di processi
- Tempo $O(n)$ per selezionare il processo, dipende dal numero di processi
- Metodo non flessibile (lista unica) e poco adatto a Symmetric Multiprocessing (SMP)

Scheduling Linux prima della versione 2.5

- Prima della versione kernel 2.5 variazione dell'algoritmo di scheduling standard di UNIX
- Lo scheduler del kernel v2.4 è una estensione del Kernel v2.2.
 - Uso di quanti di tempo variabili (time slice) dipendente dalla priorità.
 - Uso di un sistema di schedulazione a crediti.
 - Sistema a crediti: un credito 10ms di esecuzione. Un credito ogni volta che si blocca. Dopo l'esecuzione di un time slice si scala un credito. No credit no CPU.
 - Quando tutti i processi in stato RUNNING hanno esaurito i crediti, il kernel ricalcola il credito per tutti i processi (recrediting).
 - L'algoritmo di scheduling è $O(n)$ rispetto al numero di processi (unica lista)
- Il tempo per la selezione del processo migliore dipende dal numero di processi
- Migliorato ma non ok per SMP

Scheduling Linux versione 2.5

- Con la versione 2.5 si propone uno scheduler che lavora a tempo costante $O(1)$ indipendente dal numero di task nel sistema:
 - Preemptive e priority based
 - Due intervalli di priorità: processi time-sharing (detti **nice**) e **real-time**
 - **Real-time** da 0 a 99 e valori **nice** da 100 a 140
 - Si mappano in priorità globali con valori bassi che indicano alta priorità
 - Priorità più alte prendono time-slice q maggiori

$$\text{base time quantum (in milliseconds)} = \begin{cases} (140 - \text{static priority}) \times 20 & \text{if static priority} < 120 \\ (140 - \text{static priority}) \times 5 & \text{if static priority} \geq 120 \end{cases}$$

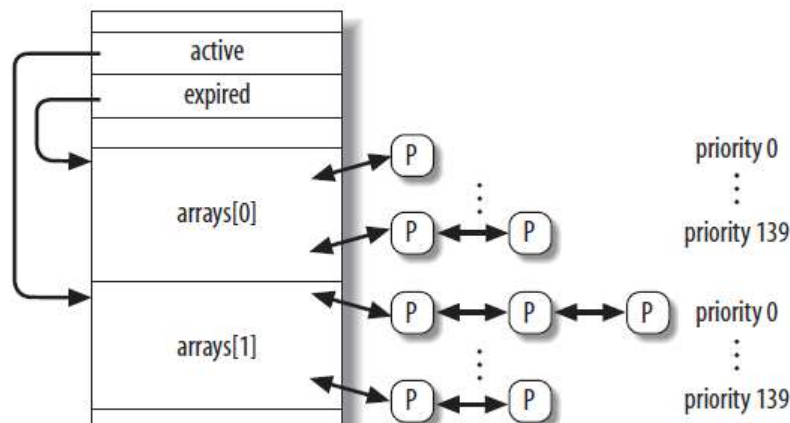
Description	Static priority	Nice value	Base time quantum
Highest static priority	100	-20	800 ms
High static priority	110	-10	600 ms
Default static priority	120	0	100 ms
Low static priority	130	+10	50 ms
Lowest static priority	139	+19	5 ms

Scheduling Linux versione 2.5

- Con la versione 2.5 si propone uno scheduler che lavora a tempo costante $O(1)$ indipendente dal numero di task nel sistema:
 - Preemptive e priority based
 - Due intervalli di priorità: processi time-sharing (detti **nice**) e **real-time**
 - **Real-time** da 0 a 99 e valori **nice** da 100 a 140
 - Si mappano in priorità globali con valori bassi che indicano alta priorità
 - Priorità più alte prendono time-slice q maggiori
 - Non goodness
- I task sono in esecuzione finché hanno il time slice (**active**)
- Quando scade non vanno in esecuzione finché tutti gli altri tasks non usano lo slice
- Tutti i task eseguibili tracciati con **code separate**
 - ▶ Due array di priorità (active, expired)
 - ▶ Task indicizzati con la priorità
 - ▶ Finiti gli attivi, gli array sono scambiati con expired (veloce scambio)
 - ▶ Scambio tempo costante, ma anche ricerca della priorità

Scheduling Linux versione 2.5

- Con la versione 2.5 si propone uno scheduler che lavora a tempo costante $O(1)$ indipendente dal numero di task nel sistema:
 - Preemptive e priority based
 - Due intervalli di priorità: processi time-sharing (detti **nice**) e **real-time**
 - **Real-time** da 0 a 99 e valori **nice** da 100 a 140
 - Si mappano in priorità globali con valori bassi che indicano alta priorità
 - Priorità più alte prendono time-slice q maggiori
 - I task sono in esecuzione finché hanno il time slice (**active**)
 - Quando scade non vanno in esecuzione finché tutti gli altri tasks non usano lo slice
 - Tutti i task eseguibili tracciati con code separate per-CPU
 - ▶ Due array di priorità (active, expired)



Scheduling Linux versione 2.5

- Con la versione 2.5 si propone uno scheduler che lavora a tempo costante $O(1)$ indipendente dal numero di task nel sistema:
 - Preemptive e priority based
 - Due intervalli di priorità: processi time-sharing (detti **nice**) e **real-time**
 - **Real-time** da 0 a 99 e valori **nice** da 100 a 140
 - Si mappano in priorità globali con valori bassi che indicano alta priorità
 - Priorità più alte prendono time-slice q maggiori
 - I task sono in esecuzione finché hanno il time slice (**active**)
 - Quando scade non vanno in esecuzione finché tutti gli altri tasks non usano lo slice
 - Tutti i task eseguibili tracciati con code separate per-CPU
 - ▶ Due array di priorità (active, expired)
 - ▶ Task indicizzati con la priorità
 - ▶ Finiti gli attivi, gli array sono scambiati con expired (veloce scambio)
 - ▶ Scambio tempo costante, ma anche ricerca della priorità
- Buon funzionamento con SMP, ma tempi di risposta non soddisfacenti per processi interattivi

Linux Scheduling in Versione 2.6.23 +

□ Completely Fair Scheduler (CFS)

- Classi di scheduling, altre possono essere aggiunte

1. Default (CFS scheduling)
2. Real-time (real-time scheduling)

□ Latenza target:

- Intervallo di tempo durante il quale un task deve andare in run almeno una volta

□ Fairness:

- se N processi uguali su latenza target T, ogni processo deve prendere T / N
- Assumiamo latenza target = 4 e processi con CPU burst 8, 4, 16, 4
- Con coda T1, T2, T3, T4 tutti eseguono 1 ms per 4 cicli ... poi T2 e T4 finiscono
- Rimane coda T1 e T3 che si dividono 2 ms per 2 cicli ... poi T1 finisce
- T3 rimane solo che fa 2 cicli da 4 ms ... e conclude

Linux Scheduling in Versione 2.6.23 +

□ Completely Fair Scheduler (CFS)

- Classi di scheduling, altre possono essere aggiunte

1. Default (CFS scheduling)
2. Real-time (real-time scheduling)

□ Latenza target:

- Intervallo di tempo durante il quale un task deve andare in run almeno una volta

□ Fairness:

- se N processi uguali su latenza target T, ogni processo deve prendere T / N

□ Priorità:

- Allocazione basata su proporzione di CPU time pesata sul peso del task
- Se N task $\text{time_slice}(\text{task}) = (\text{target_latency} / N) * \text{task_weight}$
 - ▶ Granularità minima, es. $\text{time_slice_min} = 1 \text{ ms}$

□ Vruntime:

- Il tempo scorre in modo diverso a seconda del processo in esecuzione

□ Criterio di schedulazione:

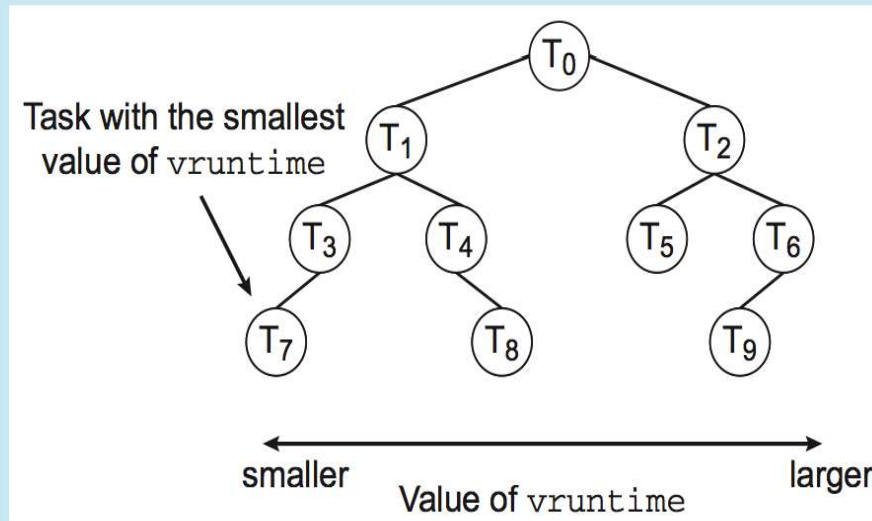
- Ad ogni invocazione selezionato il task con il vruntime più basso

Linux Scheduling in Versione 2.6.23 +

- **Completely Fair Scheduler (CFS)**
- CFS scheduler mantiene un tempo di esecuzione virtuale per ogni task in una variable **vruntime** (usata come criterio per schedulare)
 - Es. Se nice diversi (priorità diverse), per nice basso il tempo CPU scorre più lento
 - Es. Se task con stesso nice, uno I/O bound l'altro CPU bound, I/O bound non consuma il suo time_slice, CPU bound lo consuma sempre, a quel punto l'I/O bound potrà avere prelazione su CPU bound per minore vruntime
 - $$\text{vruntime} = \text{tempo_effettivo} \times \text{peso_riferimento (nice=0)} / \text{peso_task_relativo}$$
 - ▶ nice = 0 peso = 1024, nice = 10 peso = 64, nice = 19 peso = 15
 - ▶ Peso relativo rispetto al peso dei task in coda di w_i / W
- Il prossimo task lo scheduler prende il **virtual run time minore**
 - con time_slice assegnato in base al suo peso relative
- Ad un nuovo viene assegnato un min_vruntime che permetta di essere eseguito presto (ma non subito)

Performance CFS

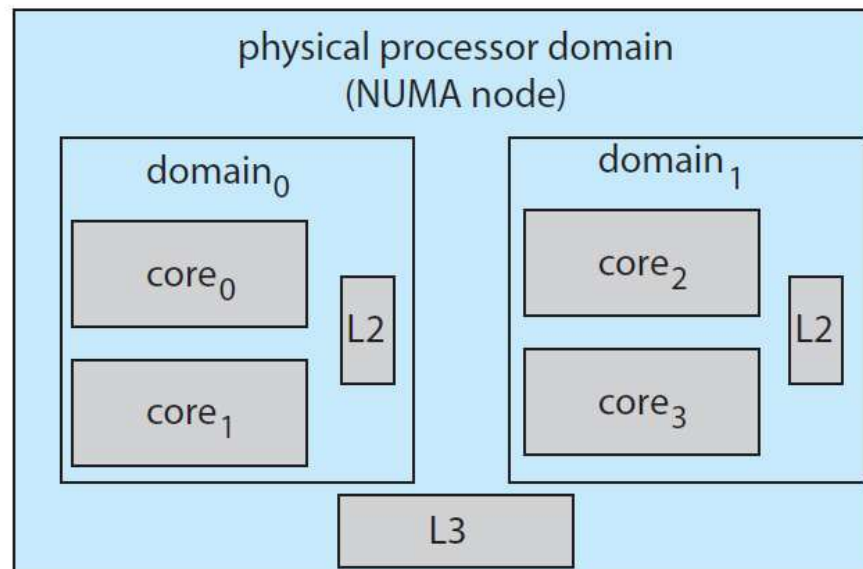
The Linux CFS scheduler provides an efficient algorithm for selecting which task to run next. Each runnable task is placed in a red-black tree—a balanced binary search tree whose key is based on the value of `vruntime`. This tree is shown below:



When a task becomes runnable, it is added to the tree. If a task on the tree is not runnable (for example, if it is blocked while waiting for I/O), it is removed. Generally speaking, tasks that have been given less processing time (smaller values of `vruntime`) are toward the left side of the tree, and tasks that have been given more processing time are on the right side. According to the properties of a binary search tree, the leftmost node has the smallest key value, which for the sake of the CFS scheduler means that it is the task with the highest priority. Because the red-black tree is balanced, navigating it to discover the leftmost node will require $O(\lg N)$ operations (where N is the number of nodes in the tree). However, for efficiency reasons, the Linux scheduler caches this value in the variable `rb_leftmost`, and thus determining which task to run next requires only retrieving the cached value.

Linux Scheduling

- ❑ Scheduler CSF supporta bilanciamento del carico, è compatibile con NUMA - riduce al minimo migrazione dei thread
- ❑ Il carico del thread è dato da una combinazione di priorità e tasso medio di utilizzo della CPU
 - ❑ Carico basso, ma alta priorità simile a carico alto e bassa priorità
 - ❑ Dalla somma dei load si ha il carico della coda che si può ribilanciare
 - ❑ Domini di scheduling per ridurre il costo del bilanciamento
 - ▶ Bilanciamento prima nel dominio più prossimo



Linux Scheduling in Versione 6.6

- ❑ Earliest Eligible Virtual Deadline First (EEVDF) scheduler (November 2023)
- ❑ Fair + deadline virtuali
- ❑ EEVDF come CFS cerca anche di ottenere un corretto uso della CPU time tra i task
- ❑ **EEVDF scheduler cerca un “eligible task” con la virtual deadline più vicina**

- ❑ EEVDF introduce due nozioni: eligibility e lag di un task.
 - ❑ EEVDF gestisce il CPU time per un task sulla base di una priorità statica (nice).
 - ❑ Ogni task ha un **virtual run time**
 - ❑ Il lag di un task è il gap tra il CPU time ideale e quello effettivo CPU per il task.
 - ❑ Lag positivo = poco CPU time allocato.
 - ❑ Lag negativo = troppo CPU time allocato (violazione dell'uso corretto di CPU time)
 - ❑ Tasks with con **lag positive** sono **eligible** (mantiene correttezza)

- ❑ EEVDF seleziona i tasks con lag positivo e calcola una virtual deadline (VD) per ognuno selezionando il task con la VD più vicina per la prossima esecuzione

- ❑ *Se CFS è orientato alla fairness pesata su priorità, EEVDF è orientato alla reattività pesata considerando priorità e deadline*

Linux Scheduling in Versione 6.6

- Earliest Eligible Virtual Deadline First (EEVDF) scheduler (November 2023)
- Fair + deadline virtuali
- EEVDF come CFS cerca anche di ottenere un corretto uso della CPU time tra i task
- **EEVDF scheduler cerca un “eligible task” con la virtual deadline più vicina**

- EEVDF introduce due nozioni: eligibility e lag di un task.
 - Esempio:
 - Assumiamo T1, T2, T3 con uguali priorità e lag iniziale 0
 - Se T1 viene schedulato per primo con 30 ms
 - Su 30 ms, l’allocazione equa sarebbe stata 10, 10, 10 (T1 quindi ha debito, T2 e T3 sono in credito) ...
dunque lag T1 = -20, lag T2 = 10, lag T3 = 10
 - Ora T1 non è più eligible, se lo scheduler assegna poi a T2 30 ms, dopo l’esecuzione i lag diventano T1 = -10, T2 = -10, T3 = 20
 - Rimane quindi solo T3 eligible

Linux Scheduling in Versione 6.6

- Earliest Eligible Virtual Deadline First (EEVDF) scheduler (November 2023)
- Fair + deadline virtuali
- EEVDF come CFS cerca anche di ottenere un corretto uso della CPU time tra i task
- **EEVDF scheduler cerca un “eligible task” con la virtual deadline più vicina**

- Per ogni processo il **virtual_run_time** scorre più o meno lento a seconda del peso del processo che è in esecuzione (veloce bassa priorità, lento alta)
 - $V(t) = V(t_{\text{prec}}) + \Delta t / w_i$
(dove w_i è il peso del task i-esimo rispetto al peso totale dei processi in coda)

- Ogni volta che lo scheduler è invocato la **virtual deadline** è aggiornata per tutti i processi in coda:
 - anche questa è calcolata pesando il tempo richiest C_i per la computazione dal task i-esimo
 - $VD = V(t) + C_i / w_i$
 - Se esiste un task con VD minore della corrente questo prelaione il precedente

Windows Scheduling

- Windows usa uno schema priority-based preemptive scheduling
 - Il thread a più alta priorità deve essere eseguito
- La parte del Kernel che si occupa di scheduling è chiamata **Dispatcher**
- Thread eseguono fino al (1) blocco, (2) fine del time slice, (3) prelazonato da un thread a priorità più alta
- I thread real-time possono prelazionare i non-real-time
- 32-livelli di schema di priorità
- **Variable class** 1-15, **real-time class** 16-31
 - Priorità 0 è il thread di gestione della memoria
- Code di thread per ogni priorità
 - Lo scheduler cerca i thread in ogni coda
 - Se non trova un runnable thread, allora lancia il **thread idle**

Windows Priority Classes

- Relazione tra le priorità nel Kernel e le priorità nelle API
- Win32 API identifica molte classi di priorità per i processi
 - REALTIME_PRIORITY_CLASS, HIGH_PRIORITY_CLASS, ABOVE_NORMAL_PRIORITY_CLASS, NORMAL_PRIORITY_CLASS, BELOW_NORMAL_PRIORITY_CLASS, IDLE_PRIORITY_CLASS
 - Tutte modificabile eccetto REALTIME
- Un thread in una priority class ha una priorità relativa
 - TIME_CRITICAL, HIGHEST, ABOVE_NORMAL, NORMAL, BELOW_NORMAL, LOWEST, IDLE
- Priority class e relative priority si combinano per formare una priorità numerica
- La priorità di base è NORMAL per la classe di appartenenza

`SetPriorityClass()`

`REALTIME_PRIORITY_CLASS—24`

`HIGH_PRIORITY_CLASS—13`

`ABOVE_NORMAL_PRIORITY_CLASS—10`

`NORMAL_PRIORITY_CLASS—8`

- Se il quantum si esaurisce la priorità si abbassa, ma mai sotto la base

Windows Priorities

	real-time	high	above normal	normal	below normal	idle priority
time-critical	31	15	15	15	15	15
highest	26	15	12	10	8	6
above normal	25	14	11	9	7	5
normal	24	13	10	8	6	4
below normal	23	12	9	7	5	3
lowest	22	11	8	6	4	2
idle	16	1	1	1	1	1

Windows Priority Classes

- Se il thread esce dal wait la priorità viene aggiornata a seconda di cosa si attendeva:
 - Es. I/O favorito, memoria meno
- Nel NORMAL_PRIORITY CLASS preferiti processi interattivi
 - Processi in foreground hanno un 3x boost di priorità rispetto a quelli in background
- Gestione dello scheduling su multiprocessori utilizzando processori logici organizzati in insiemi (SMT-set)
 - Il SO cerca di mantenere un processore ideale per un task

Valutazione Algoritmi

- Come selezionare algoritmi di CPU-scheduling per un SO?
- Determinare un criterio, poi valutare gli algoritmi
 - **Esempio 1:** massimizzare l'utilizzo della CPU, con tempo massimo di risposta inferiore a 300 millisecondi
 - **Esempio 2:** massimizzare la produttività, con tempo di completamento proporzionale (in media) al tempo di esecuzione effettivo

Valutazione Algoritmi

□ Valutazione analitica

- Definisce una formula o dati da valutare

□ Modelli deterministici

- Assume un particolare carico e definisce le performance di ogni algoritmo per quel workload
- Considera 5 processi che arrivano al tempo 0:

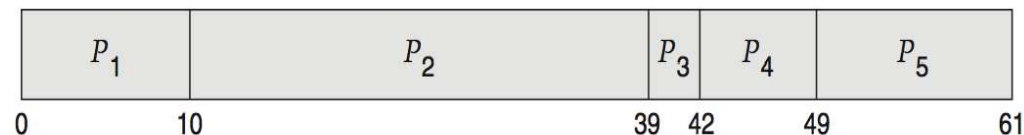
<u>Process</u>	<u>Burst Time</u>
P_1	10
P_2	29
P_3	3
P_4	7
P_5	12

- Quale algoritmo è il migliore in termini di tempi di attesa per FF, SJF e RR?

Valutazione Deterministica

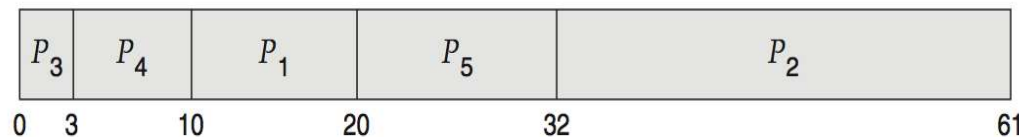
- ❓ Per ogni algoritmo calcola il minimo tempo di attesa medio
- ❓ Semplice e veloce, ma richiede numeri esatti per l'input e si applica solo a questi inputs

❓ FCFS è 28ms:

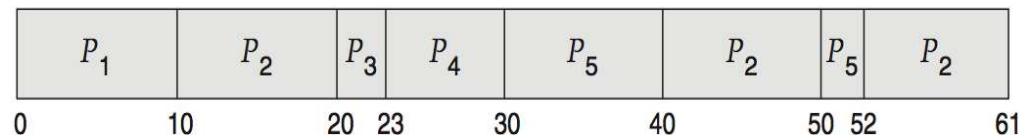


Process	Burst Time
P_1	10
P_2	29
P_3	3
P_4	7
P_5	12

❓ Non-preemptive SFJ è 13ms:

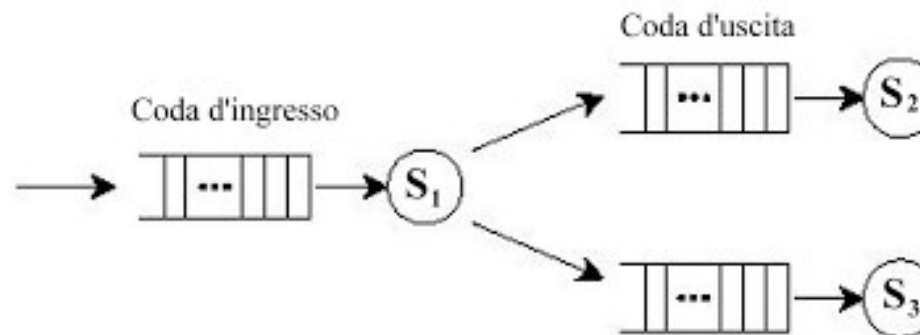


❓ RR è 23ms:



Modelli di Code

- ❑ Descrive gli arrivi dei processi, CPU e I/O burst probabilisticamente
 - ❑ Di solito distribuzioni esponenziali e calcolo di medie
 - ❑ Calcola medie di throughput, utilizzo, waiting time, etc
- ❑ Sistemi di Computer descritti come reti di server, ognuno con una coda di processi waiting
 - ❑ Tempi di arrivo e frequenze di servizio
 - ❑ Si calcola utilizzo, media lunghezza delle code, medi tempi di attesa, etc.
- ❑ Astrazione ed assunzioni ne limitano l'utilizzo



Formula di Little

- n = media lunghezza coda
- W = medio tempo di attesa in coda
- λ = media frequenza di arrivi in coda

- Legge di Little – in stato stazionario, processi che lasciano la coda devono essere pari a quelli che arrivano, quindi:
$$n = \lambda \times W$$
 - Valido per ogni algoritmo di scheduling e distribuzione degli arrivi

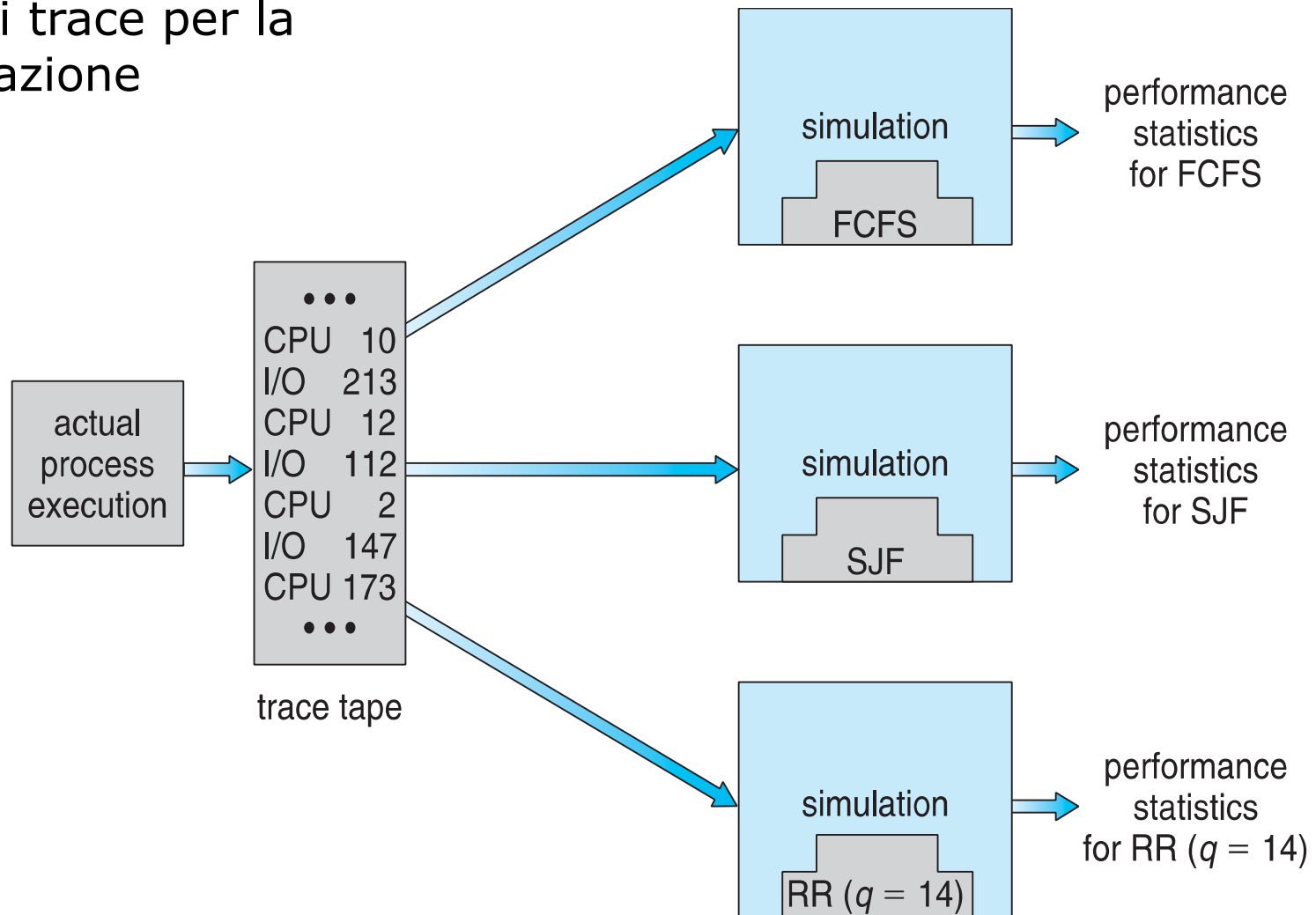
- Esempio: se in media 7 processi arrivano per secondo, e normalmente 14 processi sono in coda, allora il wait time medio per processo = 2 secondi

Simulazioni

- Modelli di code limitati, **simulazioni** più accurate
 - Modelli di sistemi di computer
 - Clock variable
 - Statistiche che indicano le performance
 - Dati per la simulazione ottenuti da
 - ▶ Generatori random
 - Distribuzioni definite matematicamente o empiricamente
 - Se empiricamente, raccolta dati su eventi reali in sistemi reali

Valuazione di Scheduler con Simulazione

Uso di trace per la simulazione



Implementazione

- Anche le simulazione hanno accuratezza limitata
- Si può direttamente implementare un nuovo scheduler e testarlo in sistemi reali
 - Alto costo, alto rischio
- Scheduler flessibili possono essere modificati per-sito o per-sistema
- API per modificare le priorità sono a disposizione